

Figure 2: Overview of strategy to use LLMs for penetration testing.

To ensure the evaluation’s fairness and accuracy, we employ several strategies. First, we involve expert-level penetration testers³ as the human testers. With their deep pentesting knowledge, these testers can precisely comprehend and execute LLM-generated operations, thus accurately assessing LLMs’ true capabilities. Second, we instruct the penetration testers to strictly execute the commands given by the LLMs, without altering any content or information, even upon identifying clear errors. They are also instructed to faithfully report the testing results back to the LLM without any additional commentary. Third, for managing UI-based operations and graphical results, we have adopted specific measures. Initially, we instruct the LLMs to minimize the use of GUI-based tools. For indispensable tools that cannot be avoided (e.g., Burp Suite), we propose a result-oriented approach: upon receiving a GUI operation instruction, the testers first execute the operation based on their expert knowledge. Subsequently, they are required to provide detailed, step-by-step textual descriptions of their actions and the observed responses at each step, which are then communicated back to the LLM. Should the LLM express any objections or comments concerning a particular step, the operation is to be repeated. This protocol ensures the integrity of the feedback loop, guaranteeing that the LLM obtains a comprehensive understanding of the testing results.

4.2 Evaluation Settings

We proceed to assess the performances of various LLMs in penetration testing tasks using the strategy mentioned above. **Model Selection.** Our study focuses on three cutting-edge LLMs that are currently accessible: GPT-3.5 with 8k token limit, GPT-4 with 32k token limit from OpenAI, and LaMDA [33] from Google. These models are selected based on their prominence in the research community and consistent availability. To interact with the LLMs mentioned above, we utilize chatbot services provided by OpenAI and Google, namely ChatGPT [34] and Bard [18]. For this paper, the terms GPT-3.5, GPT-4, and Bard will represent these three LLMs. **Experimental Setup.** Our experiments occur in a local setting with both target and testing machines on the same private network. The testing machine runs on Kali Linux [35], version 2023.1.

³We selected Offensive Security Certified Professionals (OSCP) testers.

Tool Usage. Our study aims to assess the innate capabilities of LLMs on penetration testing, without reliance on end-to-end automated vulnerability scanners such as Nexus [36] and OpenVAS [37]. Consequently, we explicitly instruct the LLMs to refrain from using these tools. We follow the LLMs’ recommendations for utilizing other tools designed to validate specific vulnerability types (e.g., sqlmap [38] for SQL injections). Occasionally, versioning discrepancies may lead the LLMs to provide incorrect instructions for tool usage. In such instances, our penetration testing experts evaluate whether the instructions would have been valid for a previous version of the tool. They then make any necessary adjustments to ensure the tool’s correct operation.

4.3 Capability Evaluation (RQ1)

To address **RQ1**, we evaluate the performance of three leading LLMs: GPT-4, Bard, and GPT-3.5. We summarize these findings in Table 1. Each LLM successfully completes at least one end-to-end penetration test, highlighting their versatility in simpler environments. Of these, GPT-4 excels, achieving success on 4 easy and 1 medium difficulty targets. Bard and GPT-3.5 follow with success on 2 and 1 easy targets, respectively. In sub-tasks, GPT-4 completes 55 out of 77 on easy targets and 30 out of 71 on medium. Bard and GPT-3.5 also show potential, finishing 16 (22.54%) and 13 (18.31%) of medium difficulty sub-tasks, respectively. However, on hard targets, all models’ performance declines. Though they can initiate the reconnaissance phase, they struggle to exploit identified vulnerabilities. This is anticipated since hard targets are designed to be especially challenging. They often feature seemingly vulnerable services that are non-exploitable, known as *rabbit holes* [39]. The pathways to exploit these machines are unique and unpredictable, resisting automated tool replication. For example, the target *Falafel* has specialized SQL injection vulnerabilities resistant to *sqlmap*. Current LLMs cannot tackle these without human expert input.

Finding 1: Large Language Models (LLMs) have shown proficiency in conducting end-to-end penetration testing tasks but struggle to overcome challenges presented by more difficult targets.

We further examine the detailed sub-task completion performances of the three LLMs compared to the walkthrough (WT), as presented in Table 2. Analyzing the completion status, we identify several areas where LLMs excel. First, they adeptly utilize common penetration testing tools to interpret the corresponding outputs, especially in enumeration tasks correctly. For example, all three evaluated LLMs successfully perform nine *Port Scanning* sub-tasks. They can configure the widely-used port scanning tool, *nmap* [40], comprehend the scan results, and formulate subsequent actions. Second, the LLMs reveal a deep understanding of prevalent vulner-

Table 1: Overall performance of LLMs on Penetration Testing Benchmark.

Tools	Easy		Medium		Hard		Average	
	Overall (7)	Sub-task (77)	Overall (4)	Sub-task (71)	Overall (2)	Sub-task (34)	Overall (13)	Sub-task (182)
GPT-3.5	1 (14.29%)	24 (31.17%)	0 (0.00%)	13 (18.31%)	0 (0.00%)	5 (14.71%)	1 (7.69%)	42 (23.07%)
GPT-4	4 (57.14%)	55 (71.43%)	1 (25.00%)	30 (42.25%)	0 (0.00%)	10 (29.41%)	5 (38.46%)	95 (52.20%)
Bard	2 (28.57%)	29 (37.66%)	0 (0.00%)	16 (22.54%)	0 (0.00%)	5 (14.71%)	2 (15.38%)	50 (27.47%)
Average	2.3 (33.33%)	36 (46.75%)	0.33 (8.33%)	19.7 (27.70%)	0 (0.00%)	6.7 (19.61%)	2.7 (20.5%)	62.3 (34.25%)

Table 2: Top 10 Types of Sub-tasks completed by each tool.

Sub-Tasks	WT	GPT-3.5	GPT-4	Bard
Web Enumeration	18	4 (22.2%)	8 (44.4%)	4 (22.2%)
Code Analysis	18	4 (22.2%)	5 (27.2%)	4 (22.2%)
Port Scanning	12	9 (75.0%)	9 (75.0%)	9 (75.0%)
Shell Construction	11	3 (27.3%)	8 (72.7%)	4 (36.4%)
File Enumeration	11	1 (9.1%)	7 (63.6%)	1 (9.1%)
Configuration Enumeration	8	2 (25.0%)	4 (50.0%)	3 (37.5%)
Cryptanalysis	8	2 (25.0%)	3 (37.5%)	1 (12.5%)
Network Enumeration	7	1 (14.3%)	3 (42.9%)	2 (28.6%)
Command Injection	6	1 (16.7%)	4 (66.7%)	2 (33.3%)
Known Exploits	6	2 (33.3%)	3 (50.0%)	1 (16.7%)

Table 3: Top Unnecessary Operations Prompted by LLMs on the Benchmark Targets

Unnecessary Operations	GPT-3.5	GPT-4	Bard	Total
Brute-Force	75	92	68	235
Exploit Known Vulnerabilities (CVEs)	29	24	28	81
SQL Injection	14	21	16	51
Command Injection	18	7	12	37

ability types, connecting them to the services on the target system. This understanding is evidenced by the successful completion of sub-tasks related to various vulnerability types. Finally, LLMs demonstrate their effectiveness in code analysis and generation, particularly in the tasks of *Code Analysis* and *Shell Construction*. These tasks require the models to read and generate codes in different programming languages. This often culminates in identifying potential vulnerabilities from code snippets and crafting the corresponding exploits. Notably, GPT-4 outperforms the other two models regarding code interpretation and generation, marking it the most suitable candidate for penetration testing tasks.

Finding 2: LLMs can efficiently use penetration testing tools, identify common vulnerabilities, and interpret source codes to identify vulnerabilities.

4.4 Comparative Analysis (RQ2)

To address **RQ2**, we examine the problem-solving strategies that LLMs employ, contrasting them with human penetration testers. In each penetration testing trial, we concentrate on

Table 4: Top causes for failed penetration testing trials

Failure Reasons	GPT3.5	GPT4	Bard	Total
Session context lost	25	18	31	74
False Command Generation	23	12	20	55
Deadlock operations	19	10	16	45
False Scanning Output Interpretation	13	9	18	40
False Source Code Interpretation	16	11	10	37
Cannot craft valid exploit	11	15	8	34

two main aspects: (1) Identifying the unnecessary operations that LLMs prompt, which are not conducive to successful penetration testing, as compared to a standard walkthrough; and (2) Understanding the specific factors that prevent LLMs from successfully executing penetration tests.

We analyze the unnecessary operations prompted by LLMs by breaking down the recorded testing procedures into sub-tasks. We employ the same method to formulate benchmark sub-tasks, as Section 3 outlines. By comparing this to a standard walkthrough, we identify the primary sub-task trials that fall outside the standard walkthrough and are thus irrelevant to the penetration testing process. The results are summarized in Table 3. We find that the most prevalent unnecessary operation prompted by LLMs is brute force. For all services requiring password authentication, LLMs typically advise brute-forcing it. This is an ineffective strategy in penetration testing. We surmise that many hacking incidents in enterprises involve password cracking and brute force. LLMs learn these reports from accident reports and are consequently considered viable solutions. Besides brute force, LLMs suggest that testers engage in CVE studies, SQL injections, and command injections. These recommendations are common, as real-world penetration testers often prioritize these techniques, even though they may not always provide the exact solution.

To understand penetration testing trial failures, we categorize the reasons for the 195 trials, as shown in Table 4. The primary failure cause is *loss of session context*. This means models often lose awareness of previous test outcomes, missing essential past results. This issue arises from LLMs’ challenge in handling conversation context. Each LLM has a fixed token window, such as GPT-4 with a capacity of 8,000 tokens [41]. If critical information for a complex task exceeds this limit, trimming it causes the loss of important details. This is problematic in intricate tests where identifying vul-

nerabilities across services and forming a cohesive exploit strategy is vital. This design flaw impacts the model’s efficacy in dealing with layered, detailed tasks.

Finding 3: LLMs struggle to maintain long-term memory, which is vital to link vulnerabilities and develop exploitation strategies effectively.

Secondly, LLMs strongly prefer the most recent tasks, adhering rigorously to a depth-first search approach. They tend to immerse deeply into resolving the issues mentioned in the most recent conversation, seldom branching out to new targets until the ongoing path is exhaustively explored. This behavior aligns with the studies [42, 43] that LLMs primarily concentrate their attention at the prompt’s beginning and end. In contrast, seasoned penetration testers adopt a more holistic approach, strategically plotting moves that promise the highest potential outcomes. When coupled with the aforementioned session context loss, this proclivity drives LLMs to become excessively anchored to one specific service. As the testing advances, the models often neglect prior discoveries, leading to an impasse.

Finding 4: LLMs strongly prefer recent tasks and a depth-first search approach, often resulting in an over-focus on one service and forgetting previous findings.

Lastly, LLMs have inaccurate result generation and hallucination issues, as noted in [44]. This phenomenon ranks as the second most frequent cause of failures and is characterized by the generation of false commands. In our study, we observe that LLMs frequently identify the appropriate tool for the task but stumble in configuring the tools with the correct settings. In some cases, they even concoct non-existent testing tools or tool modules.

Finding 5: LLMs may generate inaccurate operations or commands, often stemming from inherent inaccuracies and hallucinations.

Our exploratory study on three LLMs in penetration testing highlights their capability to complete sub-tasks. However, they face issues with long-term memory retention, reliance on a depth-first strategy, and ensuring operation accuracy. In the subsequent section, we detail our approach to mitigate these challenges and describe the design of our LLM-based penetration testing tool.

5 Methodology

5.1 Overview

In light of the challenges identified in the preceding section, we present our proposed solution, PENTESTGPT, which leverages the synergistic interplay of three LLM-powered modules.

As illustrated in Figure 3, PENTESTGPT incorporates three core modules: the **Reasoning Module**, the **Generation Module**, and the **Parsing Module**. Each module reserves one LLM session with its conversation and context. The user interacts seamlessly with PENTESTGPT, where distinct modules process different types of messages. This interaction culminates in a final decision, suggesting the subsequent step of the penetration testing process that the user should undertake. In the following sections, we elucidate our design reasoning and provide a detailed breakdown of the engineering processes behind PENTESTGPT.

5.2 Design Rationale

Our central design considerations emerged from the three challenges observed in the previous Exploratory Study (Section 4): The first challenge (*Finding 3*) pertains to the issue of penetration testing context loss due to memory retention. LLMs in their original form struggle to maintain such long-term memory due to token size limits. The second obstacle (*Finding 4*) arises from the LLM chatbots’ tendency to emphasize recent conversation content. In penetration testing tasks, this focuses on optimizing the immediate task. This approach falls short in the complex, interconnected task environment of penetration testing. The third obstacle (*Finding 5*) is tied to the inaccurate results generation by LLMs. When tasked to produce specific operations for a step in penetration testing directly, the outputs are often imprecise, sometimes even leading to false directions.

PENTESTGPT has been engineered to address these challenges, rendering it more apt for penetration testing tasks. We draw inspiration from the methodologies employed by real-world penetration testing teams, where directors plan overarching procedures, subdividing them into subtasks for individual testers. Each tester independently performs their task, reporting results without an exhaustive understanding of the broader context. The director then determines the following steps, possibly redefining tasks, and triggers the subsequent round of testing. Essentially, the director manages the overall strategy without becoming entrenched in the minutiae of the tests. This approach is mirrored in PENTESTGPT’s functionality, enhancing its efficiency and adaptability in conducting penetration tests. Our strategy divides penetration testing into two processes: identifying the next task and generating the concrete operation to complete the task. Each process is powered by one LLM session. In this setup, the LLM session responsible for task identification retains the complete context of the ongoing penetration testing status. At the same time, the generation of detailed operations and parsing of information is managed by other sessions. This division of responsibilities fosters effective task execution while preserving the overarching context.

To assist LLMs in effectively carrying out penetration testing tasks, we design a series of prompts that align with user

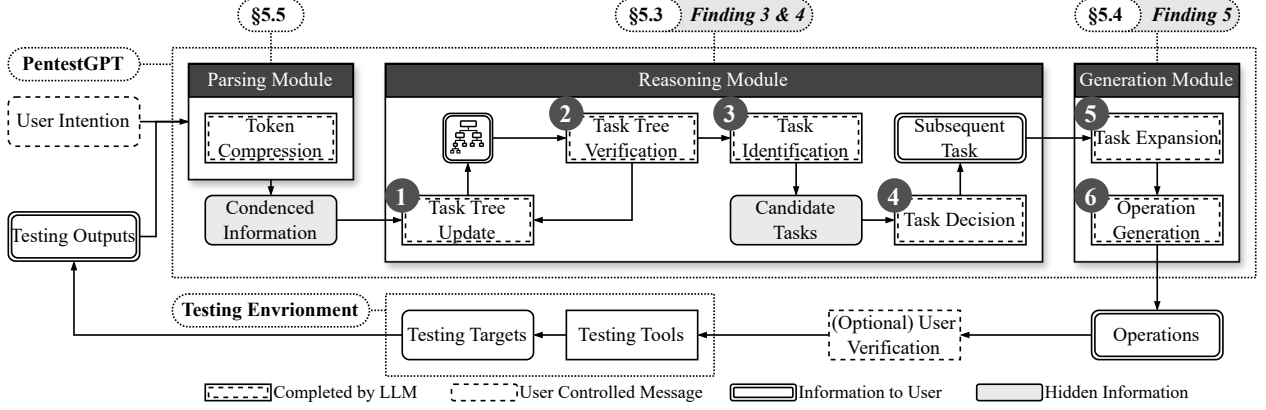


Figure 3: Overview of PENTESTGPT.

inputs. We utilize the Chain-of-Thought (CoT) [45] methodology during this process. As CoT reveals, LLMs’ performance and reasoning capabilities can be significantly enhanced using the *input, chain-of-thought, output* prompting format. Here, the *chain-of-thought* represents a series of intermediate natural language reasoning steps leading to the outcome. We dissect the penetration testing tasks into micro-steps and design prompts with examples to guide LLMs through processing penetration testing information step-by-step, ultimately leading to the desired outcomes. The complete prompts are available at our anonymized open-source project [32].

5.3 Reasoning Module

The **Reasoning Module** plays a pivotal role in our system, analogous to a team lead overseeing the penetration testing task from a macro perspective. It obtains testing results or intentions from the user and prepares the testing strategy for the next step. This testing strategy is passed to the generation module for further planning.

To effectively supervise the penetration testing process and provide precise guidance, it is crucial to translate the testing procedures and outcomes into a natural language format. Drawing inspiration from the concept of an attack tree [46], which is often used to outline penetration testing procedures, we introduce the notion of a *pentesting task tree* (PTT). This novel approach to testing status representation is rooted in the concept of an *attributed tree* [47]:

Definition 1 (Attributed Tree) *A attributed tree is an edge-labeled, attributed polytree $G = (V, E, \lambda, \mu)$ where V is a set of nodes (or vertices), E is a set of directed edges, $\lambda: E \rightarrow \Sigma$ is an edge labeling function assigning a label from the alphabet Σ to each edge and $\mu: (V \cup E) \times K \rightarrow S$ is a function assigning key(from K)-value(from S) pairs of properties to the edges and nodes.*

Given the definition of *attributed tree*, PTT is defined as follows:

Definition 2 (Pentesting Task Tree) *A PTT T is a pair (N, A) , where: (1) N is a set of nodes organized in a tree structure. Each node has a unique identifier, and there is a special node called the root that has no parent. Each node, other than the root, has exactly one parent and zero or more children. (2) A is a function that assigns to each node $n \in N$ a set of attributes $A(n)$. Each attribute is a pair (a, v) , where a is the attribute name and v is the attribute value. The set of attributes can be different for each node.*

As outlined in Figure 3, the Reasoning Module’s operation unfolds over four key steps operating over the PTT. ❶ The module begins by interpreting the user’s objectives to create an initial PTT, formatted in natural language. This involves instructing the LLM with designed prompts that contain the above PTT definition and real-world examples. The outputs from the LLM are parsed to ensure that the tree structure is correctly represented, which can be formatted in natural language through layered bullets, as shown in Figure 4. The Reasoning Module effectively overcomes the memory-loss issue by maintaining a task tree that encompasses the entire penetration testing process. ❷ After updating the tree information, a verification step is conducted on the newly updated PTT to ascertain its correctness. This process checks explicitly that only the leaf nodes of the PTT have been modified, aligning with the principle that atomic operations in the penetration testing process should only influence the status of the lowest-level sub-tasks. This step confirms the correctness of the reasoning process, safeguarding against any potential alterations to the overall tree structure due to hallucination by the LLM. If discrepancies arise, the information is reverted to the LLM for correction and regeneration. ❸ With the updated PTT, the Reasoning Module evaluates the current tree state and pinpoints viable sub-tasks that can serve as candidate

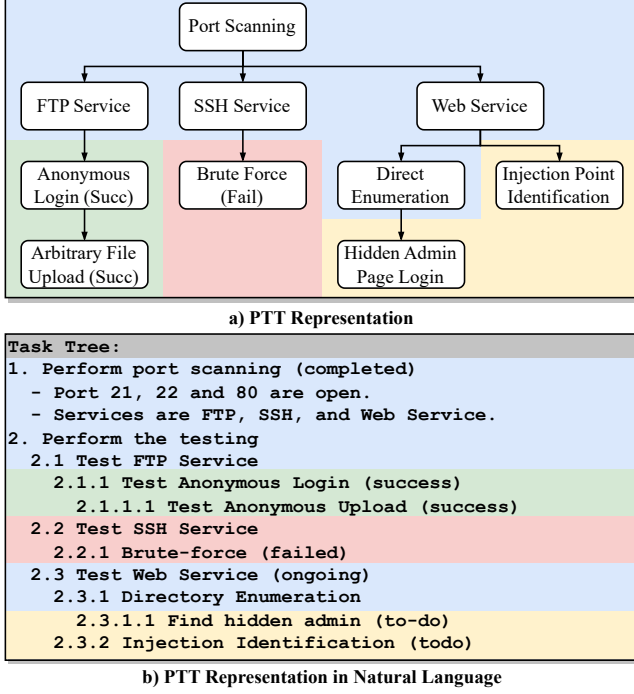


Figure 4: Pentesting Task Tree in a) visualized tree format, and b) natural language format encoded in LLM.

steps for further testing. ④ Finally, the module evaluates the likelihood of these sub-tasks leading to successful penetration testing outcomes. It then recommends the top task as the output. The expected results of this task are subsequently forwarded to the Generation Module for an in-depth analysis. This is feasible, as demonstrated in the exploratory study, since LLMs, particularly GPT-4, can identify potential vulnerabilities when provided with system status information. This procedural approach enables the Reasoning Module to address one of the inherent limitations of LLMs, precisely their tendency to concentrate solely on the most recent task. Note that in cases where the tester identifies that the correct task is incorrect or not completed in a preferred way, he could also manually revise the PTT through the interactive handle further discussed in Section 5.6.

We devise four sets of prompts to sequentially guide the Reasoning Module through the completion of each stage. To bolster the reproducibility of our results, we optimize these prompts further with a technique known as hint generation [48]. From our practical experience, we observe that LLMs are adept at interpreting the tree-structured information pertinent to penetration testing and can update it accurately in response to test outputs.

5.4 Generation Module

The Generation Module translates specific sub-tasks from the Reasoning Module into concrete commands or instructions. Each time a new sub-task is received, a fresh session is initiated in the Generation Module. This strategy effectively isolates the context of the overarching penetration task from the immediate task under execution, enabling the LLM to focus entirely on generating specific commands.

Instead of directly transforming the received sub-task into specific operations, our design employs the CoT strategy [45] to partition this process into two sequential steps. This design decision directly addresses the challenges associated with model inaccuracy and hallucination by enhancing the model’s reasoning capability. In particular, ⑤ upon the receipt of a concise sub-task from the Reasoning Module, the Generation Module begins by expanding it into a sequence of detailed steps. Notably, the prompt associated with this sub-task requires the LLM to consider the possible tools and operations available within the testing environment. ⑥ Subsequently, the Generation Module transforms each of these expanded steps into precise terminal commands ready for execution or into detailed descriptions of specific Graphical User Interface (GUI) operations to be carried out. This stage-by-stage translation eliminates potential ambiguities, enabling testers to follow the instructions directly and seamlessly. Implementing this two-step process effectively precludes the LLM from generating operations that may not be feasible in real-world scenarios, thereby improving the success rate of the penetration testing procedure.

By acting as a bridge between the strategic insights provided by the Reasoning Module and the actionable steps required for conducting a penetration test, the Generation Module ensures that high-level plans are converted into precise and actionable steps. This transformation process significantly bolsters the overall efficiency of the penetration testing procedure, and also provides human-readable outputs of the complete testing process. We present a detailed PTT generation process for a complete penetration testing target in Appendix Figure 9, accompanied by an illustrative example to aid understanding.

An Illustrative Example. We utilize a real-world running example to illuminate how the Reasoning Module and the Generation Module collaboratively operate to complete penetration testing tasks. Figure 5 illustrates a single iteration of PENTESTGPT working on the HackTheBox machine Carrier [49], a medium-difficulty target. As depicted in a-1), the PTT, in natural language format, encodes the testing status, revealing the open ports (21, 22, 80) with running services. The Reasoning Module is subsequently instructed to identify the available tasks. As highlighted in red, service scanning is the only available task on the leaf node of the PTT. This task is therefore chosen and forwarded to the Generation Module for command generation. The generated command is exe-